

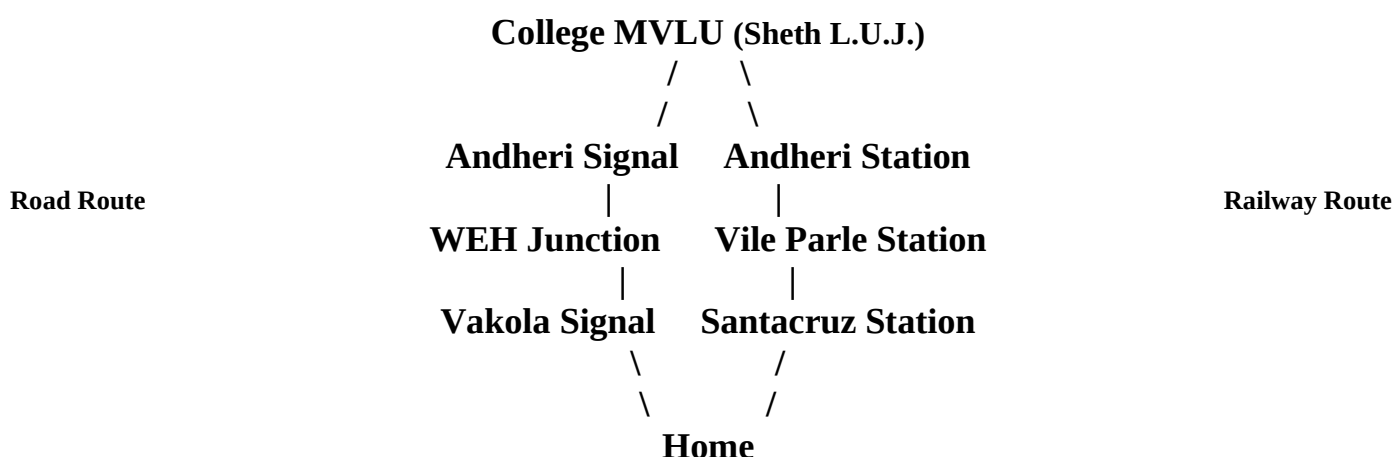
Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence and Application Development
TASK

Roll No.: T032	Name: KHUSHI
Class: TYIT	Batch:2
Date of Assignment: 22-07-26	Date/Time of Submission: 22-07-26

Problem Statement : -

The objective of this practical is to determine the optimal route from **MVLU (Sheth L.U.J.) College, Andheri East** to **Home (Vakola Pipe Line, Santacruz East)** using different Artificial Intelligence search algorithms. A graph is created to represent the possible routes between the source and destination, where each edge is assigned a travel cost (distance). The algorithms **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, **Uniform Cost Search (UCS)**, **Greedy Best-First Search**, and **A* Search** are implemented to find a path from the college to home. The performance of each algorithm is compared based on the **path found, total path cost, and number of nodes explored**, and the A* Search result is compared with the route suggested by Google Maps.

Graph Diagram : -



Edge Cost Table : -

Route 1 : Road

From	To	Cost
College	Andheri Signal	2
Andheri Signal	WEH Junction	2
WEH Junction	Vakola Signal	1
Vakola Signal	Home	2.5

Route 2 : Railway

From	To	Cost
College	Andheri Station	1.5
Andheri Station	Vile Parle Station	2
Vile Parle Station	Santacruz Station	1.5
Santacruz Station	Home	2

Total Cost of Each Route**Route 1 (Road)**

College

→ Andheri Signal

→ WEH Junction

→ Vakola Signal

(Home)

Total Cost = 2 + 2 + 1 + 2.5 = 7.5

Route 2 (Railway)

College

→ Andheri Railway Station

→ Vile Parle Station

→ Santacruz Railway Station

→ Vakola Pipe Line (Home)

Total Cost = 1.5 + 2 + 1.5 + 2 = 7

Heuristic Table (for Greedy and A*) : -

Node	h(n)
College	7.5
Andheri Signal	5.5
Andheri Station	5
WEH Junction	3
Vile Parle Station	3
Vakola Signal	2
Santacruz Station	2
Home	0

Explanation

- **Home = 0** because it is the goal node.
- As the journey gets closer to **Home**, the heuristic value decreases.
- These values help **Greedy Search** and **A*** choose the next node that appears closest to the destination.

Brief Description of Each Algorithm :-

1. Breadth-First Search (BFS)

Description:

BFS explores all nearby stations level by level using a **Queue**. It checks every possible route before moving further and finds the shortest path when all edge costs are equal.

How it worked on my route:

It started from **College**, visited **Andheri Railway Station**, then explored **Vile Parle Station** and **Western Express Metro**. Finally, it reached **Santacruz Railway Station** and **Vakola Pipe Line (Home)**.

2. Depth-First Search (DFS)

Description:

DFS explores one complete path first using a **Stack**. If the path does not reach the destination, it backtracks and tries another path.

How it worked on my route:

It started from **College** and followed one route continuously until it reached **Home**, without checking all alternative routes first.

3. Uniform Cost Search (UCS)

Description:

UCS always selects the path with the **lowest total cost** using a **Priority Queue**. It guarantees the minimum-cost path.

How it worked on my route:

It compared all possible routes from **College** and selected the route through **Andheri Railway Station** → **Vile Parle Station** → **Santacruz Railway Station** → **Home**, as it had the lowest total cost.

4. Greedy Best-First Search

Description:

Greedy Search uses only the **heuristic value (h)** and always moves to the node that appears closest to the destination.

How it worked on my route:

It selected the stations with the smallest heuristic values and quickly reached **Vakola Pipe Line (Home)** without considering the total travel cost.

5. A* Search

Description:

A* Search uses both the **path cost (g)** and the **heuristic value (h)** to find the most efficient route.

How it worked on my route:

It started from **College**, compared all available routes, and selected **College** → **Andheri Railway Station** → **Vile Parle Station** → **Santacruz Railway Station** → **Vakola Pipe Line (Home)** because it had the best combination of **lowest cost** and **shortest estimated distance**.

Python Source Code :-

1|BFS

CODE: -

```
from collections import deque
graph = {
    "College": [("Andheri Station",1)],
```

```

"Andheri Station": [("Vile Parle Station",2),("Western Express Metro",3)],
"Vile Parle Station": [("Santacruz Station",2)],
"Western Express Metro": [("Kalina",2)],
"Kalina": [("Santacruz Station",2)],
"Santacruz Station": [("Home",1)],
"Home":[]
}

```

```

def bfs(start, goal):
    queue = deque([(start,[start],0)])
    visited = set()
    explored = 0

    while queue:
        node, path, cost = queue.popleft()

        if node in visited:
            continue

        visited.add(node)
        explored += 1

        if node == goal:
            print("Path :", " -> ".join(path))
            print("Total Cost :", cost)
            print("Nodes Explored :", explored)
            return

        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                queue.append((neighbour,path+[neighbour],cost+edge_cost))

bfs("College","Home")

```

OUTPUT:-

```

Path : College -> Andheri Station -> Vile Parle Station -> Santacruz Station -> Home
Total Cost : 6
Nodes Explored : 7

```

2]DFS

CODE: -

```

graph = {
    "College": [("Andheri Station",1)],
    "Andheri Station": [("Vile Parle Station",2),("Western Express Metro",3)],
    "Vile Parle Station": [("Santacruz Station",2)],
    "Western Express Metro": [("Kalina",2)],
    "Kalina": [("Santacruz Station",2)],
    "Santacruz Station": [("Home",1)],
    "Home":[]
}

def dfs(start, goal):
    stack=[(start,[start],0)]
    visited=set()
    explored=0

```

```

while stack:
    node,path,cost=stack.pop()

    if node in visited:
        continue

    visited.add(node)
    explored+=1

    if node==goal:
        print("Path :", " -> ".join(path))
        print("Total Cost :", cost)
        print("Nodes Explored :", explored)
        return

    for neighbour,edge_cost in reversed(graph[node]):
        if neighbour not in visited:
            stack.append((neighbour,path+[neighbour],cost+edge_cost))

```

```
dfs("College","Home")
```

OUTPUT: -

```

Path : College -> Andheri Station -> Vile Parle Station -> Santacruz Station -> Home
Total Cost : 6
Nodes Explored : 5

```

3JUCS

CODE: -

```

import heapq
graph = {
    "College": [("Andheri Station",1)],
    "Andheri Station": [("Vile Parle Station",2),("Western Express Metro",3)],
    "Vile Parle Station": [("Santacruz Station",2)],
    "Western Express Metro": [("Kalina",2)],
    "Kalina": [("Santacruz Station",2)],
    "Santacruz Station": [("Home",1)],
    "Home":[]
}

```

```

def ucs(start,goal):
    pq=[(0,start,[start])]
    visited=set()
    explored=0

    while pq:
        cost,node,path=heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        explored+=1

        if node==goal:
            print("Path :", " -> ".join(path))
            print("Total Cost :", cost)

```

```
print("Nodes Explored :", explored)
return
```

```
for neighbour,edge_cost in graph[node]:
    if neighbour not in visited:
        heapq.heappush(pq,(cost+edge_cost,neighbour,path+[neighbour]))
```

```
ucs("College","Home")
```

OUTPUT:-

```
Path : College -> Andheri Station -> Vile Parle Station -> Santacruz Station -> Home
Total Cost : 6
Nodes Explored : 6
```

4]Greedy Best-First Search

CODE: -

```
import heapq
graph = {
    "College": [("Andheri Station",1)],
    "Andheri Station": [("Vile Parle Station",2),("Western Express Metro",3)],
    "Vile Parle Station": [("Santacruz Station",2)],
    "Western Express Metro": [("Kalina",2)],
    "Kalina": [("Santacruz Station",2)],
    "Santacruz Station": [("Home",1)],
    "Home":[]
}
heuristic={
    "College":6,
    "Andheri Station":5,
    "Vile Parle Station":3,
    "Western Express Metro":4,
    "Kalina":2,
    "Santacruz Station":1,
    "Home":0
}
def greedy(start,goal):
    pq=[(heuristic[start],start,[start],0)]
    visited=set()
    explored=0
    while pq:
        h,node,path,cost=heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        explored+=1
        if node==goal:
            print("Path :", " -> ".join(path))
            print("Total Cost :", cost)
            print("Nodes Explored :", explored)
            return
        for neighbour,edge_cost in graph[node]:
            if neighbour not in visited:
                heapq.heappush(pq,(heuristic[neighbour],neighbour,path+[neighbour],cost+edge_cost))
    greedy("College","Home")
```

OUTPUT:-

```
Path : College -> Andheri Station -> Vile Parle Station -> Santacruz Station -> Home
Total Cost : 6
Nodes Explored : 5
```

5]A* Search

CODE: -

```
import heapq
graph = {
    "College": [("Andheri Station",1)],
    "Andheri Station": [("Vile Parle Station",2),("Western Express Metro",3)],
    "Vile Parle Station": [("Santacruz Station",2)],
    "Western Express Metro": [("Kalina",2)],
    "Kalina": [("Santacruz Station",2)],
    "Santacruz Station": [("Home",1)],
    "Home":[]
}
heuristic={
    "College":6,
    "Andheri Station":5,
    "Vile Parle Station":3,
    "Western Express Metro":4,
    "Kalina":2,
    "Santacruz Station":1,
    "Home":0
}
def astar(start,goal):
    pq=[(heuristic[start],0,start,[start])]
    visited=set()
    explored=0
    while pq:
        f,cost,node,path=heapq.heappop(pq)
        if node in visited:
            continue

        visited.add(node)
        explored+=1

        if node==goal:
            print("Path :", " -> ".join(path))
            print("Total Cost :", cost)
            print("Nodes Explored :", explored)
            return

        for neighbour,edge_cost in graph[node]:
            if neighbour not in visited:
                g=cost+edge_cost
                h=heuristic[neighbour]
                heapq.heappush(pq,(g+h,g,neighbour,path+[neighbour]))

    astar("College","Home")
```

OUTPUT:-

```
Path : College -> Andheri Station -> Vile Parle Station -> Santacruz Station -> Home
Total Cost : 6
Nodes Explored : 5
```

Additional Task: Google Maps Comparison with A* Search

Google Maps Route

Source: MVLU College, Dr. S. Radhakrishnan Road, Andheri East

Destination: Home (Vakola Pipe Line)

Distance: 7.5 km

Estimated Time: 21 minutes

Suggested Route:

MVLU College



Andheri East



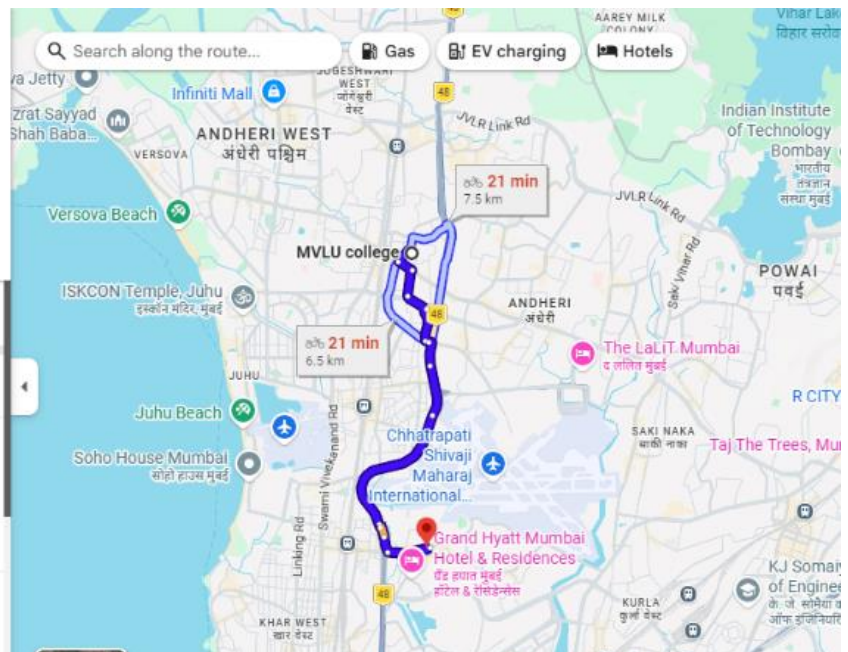
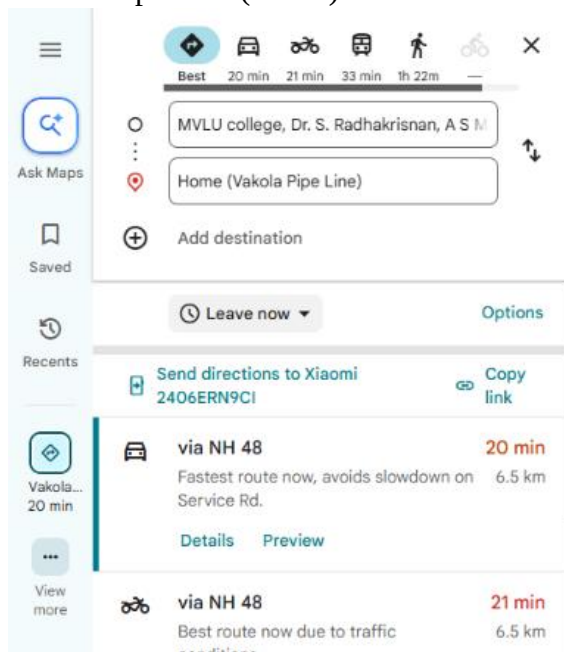
Western Express Highway (NH 48)



Vakola Junction



Vakola Pipe Line (Home)



A* Search Output

Source: College

Destination: Home

Path Found:

College



Andheri East



NH 48 (WEH)



Vakola Junction



Home

Total Cost (Distance): 6.5 km

Nodes Explored: 5

Theory Questions

1. Which algorithm would you use for the following applications?

Application	Algorithm	Reason
GPS Navigation	A* Search	Finds the shortest and fastest route using path cost and heuristic.
File System Search	DFS	Explores folders deeply before backtracking.
Maze Solving	BFS	Finds the shortest path in an unweighted maze.
Emergency Ambulance Routing	A* Search	Quickly finds the fastest route considering cost and estimated distance.
Web Crawling	BFS	Visits webpages level by level.

2. Why does changing only the data structure change the algorithm?

The data structure decides which node is explored next.

- **BFS uses a Queue (FIFO)**, so it explores nodes level by level.
- **DFS uses a Stack (LIFO)**, so it explores one path completely before returning.
- **UCS, Greedy, and A*** use a **Priority Queue**, where the next node is chosen based on priority (cost, heuristic, or both).

Therefore, even with the same graph, changing the data structure changes the search order and the final path.

3. Difference between

Shortest Path	Least-Cost Path	Fastest Path
Path with the minimum distance or fewest edges.	Path with the lowest total cost (distance, tolls, fuel, etc.).	Path that takes the least travel time considering traffic and road conditions.

4. True or False

(a) BFS always finds the least-cost path.

False.

Reason: BFS finds the path with the fewest edges, not the minimum cost when edge costs differ.

(b) DFS always finds the shortest path.

False.

Reason: DFS explores one branch deeply first and may miss a shorter path.

(c) UCS always finds the minimum-cost path.

True.

Reason: UCS expands the node with the lowest accumulated cost, guaranteeing the least-cost path.

(d) Greedy Search is always optimal.

False.

Reason: Greedy Search chooses the node with the lowest heuristic value and may ignore the overall path cost.

(e) A* Search uses both cost and heuristic values.

True.

Reason: A* calculates $f(n) = g(n) + h(n)$, where:

- $g(n)$ = actual cost from the start node.
- $h(n)$ = estimated cost to the goal. This helps A* find an optimal path efficiently.

